实　验　报　告 1

学生姓名：刘超然　　学 号：2023060904025　　指导教师：彭超

实验地点：智能自动化控制实验室　　实验时间：2025.4.19

一、实验室名称：智能自动化控制实验室

二、实验项目名称：MATLAB 的基本使用方法及系统建模实验

三、实验学时：2 学时

四、实验要求：

1. 能够熟练使用 MATLAB 基本命令，掌握 MATLAB 的 m 文件编程的基本方法、向量的表示与计算，会使用 help 命令。
2. 能够熟练使用 MATLAB 语言建立系统的不同形式的数学模型，以及模型之间的转换，熟悉系统的多种模型形式；能够熟练使用 MATLAB 语言命令进行系统的多种连接，熟悉控制系统的结构化简。

五、实验目的：

1. MATLAB 的数据表示，基本运算、程序控制和曲线绘图等语句；
2. MATLAB 程序设计的基本方法；
3. 使用 MATLAB 进行传递函数、零极点增益和状态空间模型三种系统模型的建立和转换方法；
4. 使用 MATLAB 命令实现系统的串联、并联、反馈连接以及控制系统结构化简；

六、实验内容：

习题 1 试用两种不同的指令，在闭区间[0,2pi]上产生具有 10 个等距采样点的一维数组，并用 size 和 length 命令返回向量维度。（提示：冒号生成法，linspace 命令）。

习题 2 分别用 >, 和~= 求得 a=[5:1:15] 与 b=[1 2 8 8 7 10 12 11 13 14 15]的比较, 并对结果进行分析.

习题 3 绘制[0，4π]区间上的 x1=10sint 曲线，并要求：
（1）颜色为红色、数据点标记为加号；

（2）坐标轴控制：显示范围、刻度线、比例、网络线
（3）标注控制：坐标轴名称、标题、相应文本；

习题 4 已知系统传递函数为：

$$G(s)=\frac{4s+1}{s^4+s^3+s^2+2s+1}$$

使用MATLAB表示其传递函数模型，并转换为零极点模型和状态空间模型。

习题 5 已知系统传递函数为

$$G(s)=\frac{2(s+4)(s+5)}{(s+1)(s+2)(s+3)}$$

使用MATLAB建立其零极点模型，并转换为传递函数模型和状态空间模型。

习题 6 已知一负反馈系统的前向通道和反馈通道传递函数分别为：

$$G(s)=\frac{5s+12}{(s+0.5)(s+10)}，\ H(s)=\frac{s+3}{s^2+7s+5}$$

求系统开环和闭环传递函数。

七、实验器材（设备、元器件）：

计算机

八、实验步骤：

实验代码：

习题 1：

```
x=[0:2*pi/9:2*pi];
disp('使用冒号生成法的结果：');
disp(x);
disp(['size:',mat2str(size(x))]);
disp(['length:',num2str(length(x))]);
y=linspace(0,2*pi,10);
disp('使用 linspace 生成法的结果：');
disp(y);
disp(['size:',mat2str(size(y))]);
disp(['length:',num2str(length(y))]);
```

习题 2：

```
a=[5:1:15];
b=[1 2 8 8 7 10 12 11 13 14 15];
result1=a>b;
```

```
disp('a>b 的结果');
disp(result1);
result2=a~=b;
disp('a~=b 的结果');
disp(result2);
```

习题 3：

```
t=[0:0.1:4*pi];
x1=10*sin(t);
plot(t,x1,'r+-');
xlim([0,4*pi]);
ylim([-12,12]);
xticks(0:pi:4*pi);
yticks(-10:5:10);
axis equal;
grid on;
xlabel('时间t');
ylabel('函数 x1=10sint');
title('x1=10sint 曲线图');
text(pi,10,'峰值点','FontSize',10);
```

习题 4：

```
num=[4 1];
den=[1 1 1 2 1];
G=tf(num,den);
disp('传递函数 G(s) 为:');
disp(G);
Z=zpk(G);
disp('零极点模型 G(zpk) 为:');
disp(Z);
S=ss(G);
disp('状态空间模型 G(ss) 为:');
disp(S);
```

习题 5：

```
z=[-4,-5];
p=[-1,-2,-3];
k=2;
Z=zpk(z,p,k);
disp('零极点模型 G(zpk) 为:');
disp(Z);
G=tf(Z);
disp('传递函数模型 G(s) 为:');
disp(G);
```

```
S=ss(Z);
disp('状态空间模型 G(ss) 为:');
disp(S);
```

习题 6：

```
G_num=[5 12];
G_den=conv([1 0.5],[1 10]);
G=tf(G_num,G_den);
H_num=[1 3];
H_den=[1 7 5];
H=tf(H_num,H_den);
GH=series(G,H);
T=feedback(G,H);
disp('开环传递函数G(s)*H(s):');
disp(GH);
disp('闭环传递函数T(s):');
disp(T);
```

九、实验数据及结果分析：

习题 1：

```
>> Untitled5
使用冒号生成法的结果：
         0    0.6981    1.3963    2.0944    2.7925    3.4907    4.1888    4.8869    5.5851    6.2832

size:[1 10]
length:10
使用linspace生成法的结果：
         0    0.6981    1.3963    2.0944    2.7925    3.4907    4.1888    4.8869    5.5851    6.2832

size:[1 10]
length:10
>>
```

通过冒号生成法和 linspace 函数均成功生成[0, 2π]区间内 10 个等距采样点的一维数组。两种方式生成的向量长度均为 10，维度均为 1×10，说明两种方法均可实现等距分点功能，linspace 更加直观适用于指定分点数的场景。

习题 2：

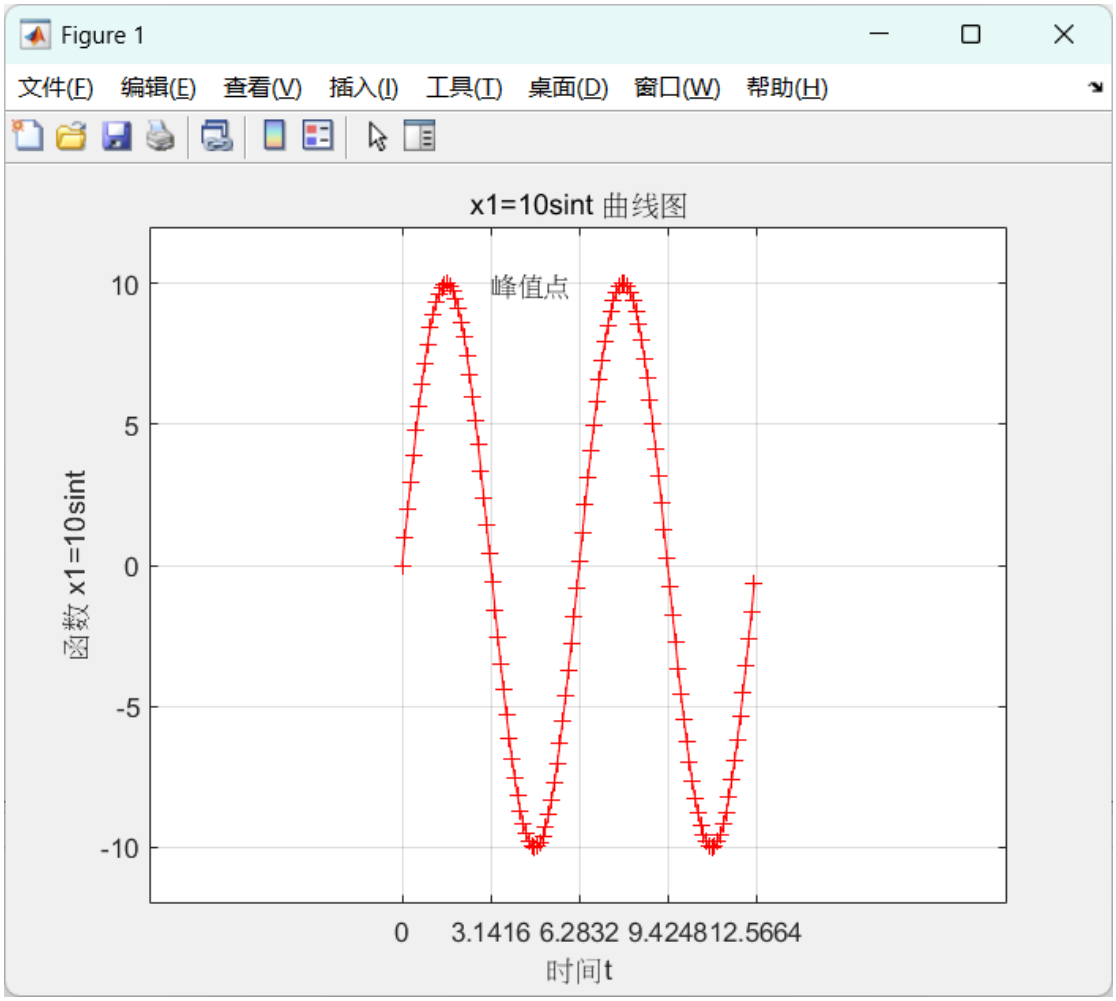
```
>> Untitled6
a>b的结果
   1    1    0    0    1    0    0    1    0    0    0

a~=b的结果
   1    1    1    0    1    0    1    1    0    0    0

>>
```

使用逻辑运算符>与~=分别对两个数组进行比较，得到的布尔数组指出了两个数组元素在相应位置的大小关系与是否相等。结果分析显示，在数组 a 中，大于 b 对应元素的位置为：第 1、2、5、6 位；不等于 b 的位置为第 1~10 位，仅最后一位元素相等。

习题 3：



绘制的函数图像x1 = 10*sin(t)符合预期，图线为红色、数据点用加号标记。坐标轴设置合理，x轴在[0, 4π]范围内分π间隔刻度，y轴在[-10, 10]间隔为5，采用等比例坐标系并加上网格。图中添加了“峰值点”注释，使图像更具信息性和可读性。

习题 4：

```
>> Untitled8
传递函数 G(s) 为:
  tf - 属性:

       Numerator: {[0 0 0 4 1]}
     Denominator: {[1 1 1 2 1]}
        Variable: 's'
         IODelay: 0
      InputDelay: 0
     OutputDelay: 0
       InputName: {''}
       InputUnit: {''}
      InputGroup: [1×1 struct]
      OutputName: {''}
      OutputUnit: {''}
     OutputGroup: [1×1 struct]
           Notes: [0×1 string]
        UserData: []
            Name: ''
              Ts: 0
        TimeUnit: 'seconds'
    SamplingGrid: [1×1 struct]

零极点模型 G(zpk) 为:
  zpk - 属性:

                Z: {[-0.2500]}
                P: {[4×1 double]}
                K: 4
    DisplayFormat: 'roots'
         Variable: 's'
          IODelay: 0
       InputDelay: 0
      OutputDelay: 0
        InputName: {''}
        InputUnit: {''}
       InputGroup: [1×1 struct]
       OutputName: {''}
       OutputUnit: {''}
      OutputGroup: [1×1 struct]
            Notes: [0×1 string]
         UserData: []
             Name: ''
               Ts: 0
         TimeUnit: 'seconds'
     SamplingGrid: [1×1 struct]

状态空间模型 G(ss) 为:
  ss - 属性:

                A: [4×4 double]
                B: [4×1 double]
                C: [0 0 2 0.5000]
                D: 0
                E: []
           Scaled: 0
        StateName: {4×1 cell}
        StatePath: {4×1 cell}
        StateUnit: {4×1 cell}
    InternalDelay: [0×1 double]
       InputDelay: 0
      OutputDelay: 0
        InputName: {''}
        InputUnit: {''}
       InputGroup: [1×1 struct]
       OutputName: {''}
       OutputUnit: {''}
      OutputGroup: [1×1 struct]
            Notes: [0×1 string]
         UserData: []
             Name: ''
               Ts: 0
         TimeUnit: 'seconds'
     SamplingGrid: [1×1 struct]

fx >>
```

成功将传递函数G(s) = (4s + 1)/(s^4 + s^3 + s^2 + 2s + 1)表示为传递函数模型，并成功转换为零极点模型（zpk）和状态空间模型（ss）。说明掌握了系统模型之间的转换方法，验证了MATLAB中tf、zpk、ss等命令的应用。

习题 5：

```
>> Untitled9
零极点模型 G(zpk) 为:
  zpk - 属性:

                Z: {[2×1 double]}
                P: {[3×1 double]}
                K: 2
    DisplayFormat: 'roots'
         Variable: 's'
          IODelay: 0
       InputDelay: 0
      OutputDelay: 0
        InputName: {''}
        InputUnit: {''}
       InputGroup: [1×1 struct]
       OutputName: {''}
       OutputUnit: {''}
      OutputGroup: [1×1 struct]
            Notes: [0×1 string]
         UserData: []
             Name: ''
               Ts: 0
         TimeUnit: 'seconds'
     SamplingGrid: [1×1 struct]

传递函数模型 G(s) 为:
  tf - 属性:

       Numerator: {[0 2 18 40]}
     Denominator: {[1 6 11 6]}
        Variable: 's'
         IODelay: 0
      InputDelay: 0
     OutputDelay: 0
       InputName: {''}
       InputUnit: {''}
      InputGroup: [1×1 struct]
      OutputName: {''}
      OutputUnit: {''}
     OutputGroup: [1×1 struct]
           Notes: [0×1 string]
        UserData: []
            Name: ''
              Ts: 0
        TimeUnit: 'seconds'
    SamplingGrid: [1×1 struct]

状态空间模型 G(ss) 为:
  ss - 属性:

                A: [3×3 double]
                B: [3×1 double]
                C: [0.8660 0.8660 0.5000]
                D: 0
                E: []
           Scaled: 0
        StateName: {3×1 cell}
        StatePath: {3×1 cell}
        StateUnit: {3×1 cell}
    InternalDelay: [0×1 double]
       InputDelay: 0
      OutputDelay: 0
        InputName: {''}
        InputUnit: {''}
       InputGroup: [1×1 struct]
       OutputName: {''}
       OutputUnit: {''}
      OutputGroup: [1×1 struct]
            Notes: [0×1 string]
         UserData: []
             Name: ''
               Ts: 0
         TimeUnit: 'seconds'
     SamplingGrid: [1×1 struct]

fx >>
```

已知零极点模型参数后，成功通过zpk函数构建零极点模型，再通过tf和ss转换为传递函数模型和状态空间模型。转换过程中，保持系统描述一致，结果正确，表明掌握了不同模型之间的互通关系。

习题 6：

```
>> Untitled10
开环传递函数G(s)*H(s):
  tf - 属性:

      Numerator: {[0 0 5 27 36]}
    Denominator: {[1 17.5000 83.5000 87.5000 25]}
       Variable: 's'
        IODelay: 0
     InputDelay: 0
    OutputDelay: 0
      InputName: {''}
      InputUnit: {''}
     InputGroup: [1×1 struct]
     OutputName: {''}
     OutputUnit: {''}
    OutputGroup: [1×1 struct]
          Notes: [0×1 string]
       UserData: []
           Name: ''
             Ts: 0
       TimeUnit: 'seconds'
   SamplingGrid: [1×1 struct]

闭环传递函数T(s):
  tf - 属性:

      Numerator: {[0 5 47 109 60]}
    Denominator: {[1 17.5000 88.5000 114.5000 61]}
       Variable: 's'
        IODelay: 0
     InputDelay: 0
    OutputDelay: 0
      InputName: {''}
      InputUnit: {''}
     InputGroup: [1×1 struct]
     OutputName: {''}
     OutputUnit: {''}
    OutputGroup: [1×1 struct]
          Notes: [0×1 string]
       UserData: []
           Name: ''
             Ts: 0
       TimeUnit: 'seconds'
   SamplingGrid: [1×1 struct]

fx >>
```

通过series命令得到开环传递函数G(s)H(s)，再通过feedback命令求得闭环传递函数T(s)，符合控制理论中负反馈系统模型推导方法。进一步巩固了MATLAB在系统结

构建模方面的实用性。

十、实验结论：

本次实验通过一系列练习，掌握了MATLAB的基本命令、向量操作、函数绘图及系统建模方法，特别是三种常见系统模型（传递函数、零极点、状态空间）之间的建立与相互转换。通过MATLAB命令对系统连接与简化进行仿真操作，有效提升了对控制系统建模及分析的理解能力，为后续控制工程课程学习奠定了坚实基础。

十一、总结及心得体会：

通过本次实验，我深入掌握了MATLAB基本操作与编程方法，也提升了对控制系统建模与转换的实际操作能力。特别是对于复杂系统的传递函数建立及多种模型的灵活切换，有了更直观的理解。在绘图过程中体会到了MATLAB强大的可视化功能，对数据结构、系统行为有了更清晰的认识。整个实验过程逻辑清晰，操作步骤规范，充分体现了理论与实践相结合的教学目标。

十二、对本实验过程及方法、手段的改进建议：

1. 增加 MATLAB 命令讲解深度：实验中大量使用 MATLAB 命令进行系统建模与转换，建议在实验指导书中提供更详细的命令说明与常见错误提示，特别是 tf、zpk、ss、feedback 等命令的使用细节。

2. 实验内容层次可更优化：实验中的题目数量较多，部分题目重复性较强，建议根据知识点将实验内容划分为基础命令练习、系统建模转换和结构化简三个部分，增强逻辑性，避免学生陷入机械操作。

3. 引入图形化工具 Simulink：在 MATLAB 编程的基础上，可考虑引入 Simulink 模块的入门操作，使学生对系统结构的直观理解更加清晰，提升对复杂系统的认知能力。

报告评分：

指导教师签字：